# Project Genesis Implementation Plan

## Overview

This document outlines the technical implementation required to bring the Project Genesis workflow to life within Cepho. The implementation transforms the proven voice-to-expert-team process into an automated, repeatable system.

---

## Phase 1: Voice Note Integration

### Objective

Enable users to start projects with a voice note that pre-populates the Project Genesis wizard.

### Technical Requirements

**Frontend Changes:**

1. Add “Start with Voice Note” button to Project Genesis entry point
2. Create VoiceNoteIntake component with recording interface
3. Display transcription in real-time as user speaks
4. Show “Analyzing…” state while extracting project parameters

**Backend Changes:**

1. Create `/api/project-genesis/transcribe` endpoint
2. Integrate with existing Whisper API for transcription
3. Create `/api/project-genesis/analyze` endpoint for parameter extraction
4. Use LLM to extract structured data from transcription

**Data Model:**

```
interface ProjectIntake {
  id: string;
  voiceNoteUrl: string;
  transcription: string;
  extractedParams: {
    objective: string;
    targetAudience: string;
    isInternal: boolean;
    timeline: string;
    successCriteria: string;
    constraints: string[];
    deliverableFormat: string;
  };
  confidence: number;
  createdAt: Date;
}
```

## Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 1.1 | Create VoiceNoteIntake component | 4 hours |
| 1.2 | Build transcription API endpoint | 2 hours |
| 1.3 | Build parameter extraction with LLM | 4 hours |
| 1.4 | Integrate with Project Genesis wizard | 3 hours |
| 1.5 | Add pre-population logic | 2 hours |
| 1.6 | Testing and refinement | 3 hours |

**Total: 18 hours**

---

# Phase 2: Project Genesis Wizard Enhancement

## Objective

Enhance the wizard to accept pre-populated values and allow user confirmation or refinement.

## Technical Requirements

**Frontend Changes:**

1. Update ProjectGenesisWizard to accept initial values
2. Add “Extracted from voice note” indicators
3. Allow inline editing of pre-populated fields
4. Add confidence indicators for extracted values

**Wizard Flow:**

```
Voice Note → Transcription → Extraction → Pre-populated Wizard → User
Confirmation → Project Created
```

## Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 2.1 | Update wizard state management | 2 hours |
| 2.2 | Add pre-population indicators | 2 hours |
| 2.3 | Implement inline editing | 2 hours |
| 2.4 | Add confidence display | 1 hour |
| 2.5 | Testing | 2 hours |

**Total: 9 hours**

---

# Phase 3: Expert Team Assembly

## Objective

Automatically assemble a 12-person expert team based on project type and requirements.

## Technical Requirements

**Backend Changes:**

1. Create `/api/project-genesis/suggest-team` endpoint
2. Implement team template selection logic
3. Create expert matching algorithm
4. Build team composition validator

**Algorithm Logic:**

```typescript
function suggestTeam(projectBrief: ProjectBrief): ExpertTeam {
  // 1. Classify project type
  const projectType = classifyProject(projectBrief);

  // 2. Select base template
  const template = getTemplate(projectType);

  // 3. Identify target audience
  const audience = identifyAudience(projectBrief);

  // 4. Select audience voices
  const audienceExperts = selectAudienceVoices(audience);

  // 5. Select domain specialists
  const domainExperts = selectDomainExperts(projectBrief, template);

  // 6. Assign project lead
  const projectLead = assignProjectLead(domainExperts, projectType);

  // 7. Include Digital Twin
  const digitalTwin = getDigitalTwin();

  // 8. Assemble and validate
  return assembleTeam([projectLead, ...audienceExperts, ...domainExperts,
digitalTwin]);
}
```

**Frontend Changes:**

1. Create TeamAssemblyView component
2. Display suggested team with expert cards
3. Allow user to swap experts
4. Show team composition rationale

### Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 3.1 | Create team suggestion endpoint | 4 hours |
| 3.2 | Implement project classification | 3 hours |
| 3.3 | Build expert matching algorithm | 4 hours |
| 3.4 | Create TeamAssemblyView component | 4 hours |
| 3.5 | Add expert swap functionality | 2 hours |
| 3.6 | Testing | 3 hours |

**Total: 20 hours**

---

## Phase 4: Project Lead Assignment

### Objective

Automatically assign a Project Lead based on project type with user override capability.

### Technical Requirements

**Backend Changes:**

1. Create project lead assignment logic
2. Store project lead in project record
3. Create lead notification system

**Assignment Rules:**

| Project Type | Default Lead |
|---|---|
| Learning & Development | L&D Expert (GCC or Western based on context) |
| Investment Analysis | Warren Buffett persona |
| Marketing & Brand | Brand Strategy Expert |
| Technology & Product | Tech Strategy Expert |
| Legal & Compliance | Legal Expert |
| Healthcare & Wellness | Healthcare Expert |

**Frontend Changes:**

1. Display assigned lead prominently
2. Add “Change Lead” option
3. Show lead’s coordination responsibilities

## Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 4.1 | Create lead assignment logic | 2 hours |
| 4.2 | Update project data model | 1 hour |
| 4.3 | Build lead display component | 2 hours |
| 4.4 | Add lead change functionality | 1 hour |
| 4.5 | Testing | 1 hour |

**Total: 7 hours**

---

# Phase 5: Research Coordination

## Objective

Enable the expert team to conduct parallel research and synthesize findings.

## Technical Requirements

**Backend Changes:**

1. Create `/api/project-genesis/initiate-research` endpoint
2. Build parallel research orchestration
3. Create research synthesis logic
4. Store research outputs per expert

**Research Flow:**

```
Project Brief → Expert Assignment → Parallel Research → Individual Reports →
Synthesis → User Review
```

**Data Model:**

```typescript
interface ExpertResearch {
  projectId: string;
  expertId: string;
  domain: string;
  findings: string;
  recommendations: string[];
  sources: string[];
  confidence: number;
  completedAt: Date;
}

interface ResearchSynthesis {
  projectId: string;
  leadExpertId: string;
  summary: string;
  keyFindings: string[];
  gaps: string[];
  recommendations: string[];
  createdAt: Date;
}
```

**Frontend Changes:**

1. Create ResearchProgressView component
2. Display individual expert progress
3. Show synthesis when complete
4. Allow user to request deeper research

### Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 5.1 | Create research initiation endpoint | 3 hours |
| 5.2 | Build parallel research orchestration | 4 hours |
| 5.3 | Implement synthesis logic | 4 hours |
| 5.4 | Create ResearchProgressView | 3 hours |
| 5.5 | Add deeper research request | 2 hours |
| 5.6 | Testing | 3 hours |

**Total: 19 hours**

---

## Phase 6: Deliverable Creation

### Objective

Enable the expert team to create deliverables based on research synthesis.

### Technical Requirements

**Backend Changes:**

1. Create `/api/project-genesis/create-deliverable` endpoint
2. Build deliverable generation logic
3. Implement quality gate workflow
4. Store deliverable versions

**Deliverable Types:**

| Format | Generation Method |
|---|---|
| Presentation | LLM generates slide content, template applied |
| Report | LLM generates sections, formatting applied |
| Executive Summary | LLM synthesizes key points |
| Implementation Plan | LLM structures actions and timeline |

**Quality Gates:**

1. Storytelling review (narrative coherence)
2. Visual review (design quality)
3. Digital Twin review (user alignment)
4. Audience review (target resonance)

**Frontend Changes:**

1. Create DeliverablePreview component
2. Display quality gate status
3. Allow inline feedback
4. Show revision history

## Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 6.1 | Create deliverable generation endpoint | 4 hours |
| 6.2 | Build quality gate workflow | 3 hours |
| 6.3 | Create DeliverablePreview component | 4 hours |
| 6.4 | Add feedback and revision system | 3 hours |
| 6.5 | Testing | 3 hours |

**Total: 17 hours**

---

# Phase 7: User Review and Iteration

## Objective

Enable users to review deliverables and provide feedback for iteration.

## Technical Requirements

**Frontend Changes:**

1. Create ReviewInterface component
2. Add voice note feedback option
3. Display revision comparison
4. Show approval workflow

**Feedback Methods:**

1. Voice note (transcribed and analyzed)
2. Text comments (inline or general)
3. Direct annotation (on presentations/documents)

**Backend Changes:**

1. Create `/api/project-genesis/submit-feedback` endpoint
2. Build feedback analysis logic
3. Create revision tracking system

### Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 7.1 | Create ReviewInterface component | 4 hours |
| 7.2 | Add voice feedback option | 2 hours |
| 7.3 | Build feedback analysis | 3 hours |
| 7.4 | Create revision tracking | 2 hours |
| 7.5 | Testing | 2 hours |

**Total: 13 hours**

---

## Phase 8: Workflow Integration

### Objective

Integrate Project Genesis with existing Workflow page for status tracking.

### Technical Requirements

**Frontend Changes:**

1. Add Project Genesis projects to Workflow view
2. Display phase indicators (Intake → Research → Assembly → Creation → Review → Complete)
3. Show expert team activity
4. Add project timeline

**Backend Changes:**

1. Create project status tracking
2. Build activity logging
3. Create notification triggers

### Implementation Steps

| Step | Task | Effort |
|---|---|---|
| 8.1 | Add to Workflow view | 3 hours |
| 8.2 | Create phase indicators | 2 hours |
| 8.3 | Build activity logging | 2 hours |
| 8.4 | Add notifications | 2 hours |
| 8.5 | Testing | 2 hours |

**Total: 11 hours**

---

## Implementation Summary

| Phase | Description | Effort |
|---|---|---|
| 1 | Voice Note Integration | 18 hours |
| 2 | Wizard Enhancement | 9 hours |
| 3 | Expert Team Assembly | 20 hours |
| 4 | Project Lead Assignment | 7 hours |
| 5 | Research Coordination | 19 hours |
| 6 | Deliverable Creation | 17 hours |
| 7 | User Review and Iteration | 13 hours |
| 8 | Workflow Integration | 11 hours |

**Total Implementation Effort: 114 hours**

---

# Priority Recommendations

## Immediate (Week 1)

Focus on Phase 1 (Voice Note Integration) and Phase 2 (Wizard Enhancement). This delivers the core value of voice-to-wizard pre-population.

## Short-term (Week 2-3)

Implement Phase 3 (Expert Team Assembly) and Phase 4 (Project Lead Assignment). This enables automated team formation.

## Medium-term (Week 4-6)

Complete Phase 5 (Research Coordination) and Phase 6 (Deliverable Creation). This enables end-to-end project execution.

## Final (Week 7-8)

Finish Phase 7 (User Review) and Phase 8 (Workflow Integration). This completes the feedback loop and visibility.

---

# Success Metrics

| Metric | Target |
|---|---|
| Voice note to pre-populated wizard | Under 60 seconds |
| Pre-population accuracy | 80%+ fields correct |
| Team assembly time | Under 30 seconds |
| Research initiation to synthesis | Under 30 minutes |
| First draft delivery | Within user timeline |
| User satisfaction | 90%+ approval rate |

---

*This implementation plan transforms the proven workflow into a systematic, automated process within Cepho.*